SQL

SQL Query Optimization - Quick Notes

1. Analyze Execution Plan – Identify table scans and costly operations.
2. Use Proper Indexing – Index WHERE, JOIN, ORDER BY columns.
3. Avoid SELECT * – Fetch only required columns.
4. Write SARGable Queries – Avoid functions on columns.
5. Optimize Joins – Use correct joins and index join columns.
6. Use EXISTS instead of IN – Better for large datasets.
7. Filter Data Early – Reduce dataset size using WHERE clause.
8. Avoid Functions in WHERE – Prevents index usage.
9. Use Window Functions – ROW_NUMBER, RANK efficiently.
10. Avoid Cursors – Prefer set-based operations.
11. Use CTE/Temp Tables Wisely – Simplify complex queries.
12. Monitor Performance – Use execution plan & statistics.

One-Line Answer: I optimize SQL queries by reducing I/O, using indexes effectively, and analyzing execution plans

Top 20 SQL Tricky Interview Questions with Answers

🔥 1. Difference between WHERE and HAVING

Answer:

WHERE → filters **before grouping**

HAVING → filters **after GROUP BY**

🔥 2. Find 2nd Highest Salary

```
SELECT MAX(Salary)
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

🔥 3. Find Duplicate Records

```
SELECT Name, COUNT(*)
FROM Employees
GROUP BY Name
HAVING COUNT(*) > 1;
```

🔥 4. Delete Duplicate Records (Keep One)

```
WITH CTE AS (
SELECT *, ROW_NUMBER() OVER (PARTITION BY Name ORDER BY Id)
AS rn
FROM Employees
)
DELETE FROM CTE WHERE rn > 1;
```

🔥 5. Difference between INNER JOIN and LEFT JOIN

Answer:

INNER JOIN → only matching rows

LEFT JOIN → all left + matching right

🔥 6. Find Employees Who Don’t Have Manager

```
SELECT *
FROM Employees
WHERE ManagerId IS NULL;
```

🔥 7. Find Nth Highest Salary (Generic)

```
SELECT Salary
FROM (
SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) rnk
FROM Employees
) t
WHERE rnk = 3;
```

🔥 8. Difference between DELETE, TRUNCATE, DROP

Answer:

DELETE → row-by-row, rollback possible

TRUNCATE → fast, no rollback

DROP → removes table completely

🔥 9. What is Index? Types?

Answer:

Improves query speed

Types: Clustered, Non-clustered

🔥 10. Find Employees with Same Salary

```
SELECT Salary
FROM Employees
GROUP BY Salary
HAVING COUNT(*) > 1;
```

🔥 11. What is a View?

Answer:

Virtual table based on query

No physical data storage

🔥 12. Difference between UNION and UNION ALL

Answer:

UNION → removes duplicates

UNION ALL → keeps duplicates (faster)

🔥 13. Find Top 3 Salaries

```
SELECT DISTINCT TOP 3 Salary
FROM Employees
ORDER BY Salary DESC;
```

🔥 14. Self Join Example (Employee-Manager)

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerId = m.Id;
```

🔥 15. What is Normalization?

Answer:

Process to reduce redundancy

Improves data integrity

🔥 16. What is Denormalization?

Answer:

Opposite of normalization

Improves read performance

🔥 17. Difference between RANK() and DENSE_RANK()

Answer:

RANK → skips numbers

DENSE_RANK → no gaps

🔥 18. Find Missing Numbers in Sequence

```
SELECT t1.Id + 1 AS MissingNumber
FROM Numbers t1
LEFT JOIN Numbers t2 ON t1.Id + 1 = t2.Id
WHERE t2.Id IS NULL;
```

🔥 19. What is a Stored Procedure?

Answer:

Precompiled SQL code

Improves performance & reuse

🔥 20. What is ACID Properties?

Answer:

Atomicity

Consistency

Isolation

Durability

✅ SQL Query Optimization

1. Analyze Execution Plan

Check **table scan vs index seek**

Identify costly operations

2. Use Proper Indexing

Create indexes on:

- WHERE columns
- JOIN columns
- ORDER BY columns

3. Avoid SELECT *

Fetch only required columns

Reduces I/O and memory usage

4. Write SARGable Queries

Avoid functions on columns

Example: avoid YEAR(DateColumn)

5. Optimize Joins

Use correct joins (INNER, LEFT)

Ensure join columns are indexed

6. Use EXISTS instead of IN (for large data)

Better performance in many cases

7. Filter Data Early

Apply WHERE conditions early

Reduce dataset size

8. Avoid Functions in WHERE Clause

Functions prevent index usage

9. Use Proper Aggregations / Window Functions

Use ROW_NUMBER(), RANK() efficiently

10. Avoid Cursors

Prefer **set-based operations**

11. Use CTE / Temp Tables Wisely

Break complex queries

Improve readability & performance

12. Monitor and Tune Regularly

Use:

- Execution plan
- Query statistics